# Hanzo Observability Stack: Search, Telemetry, and Session Analytics for Production Systems

Zach Kelling

*Hanzo AI*

2018

**Abstract**

We present the Hanzo observability stack, comprising four components developed between 2016 and 2018: a full-text search engine (`hanzo/search`, April 2018, 16,000 commits), a telemetry and metrics pipeline (`hanzo/telemetry`, June 2018, 8,200 commits), a session recording system (`hanzo/rrweb`, September 2018, 1,600 commits), and an event analytics platform for funnel analysis and user behavior tracking. An OpenAPI specification layer (2016) provides contract-first API documentation across all components. This infrastructure became the foundation for Hanzo Insights, the privacy-first analytics platform.

# 1 Introduction

Observability in production systems requires three pillars: logs, metrics, and traces. For user-facing applications, a fourth pillar is equally important: behavioral analytics—understanding how users interact with the product. The Hanzo observability stack addresses all four requirements with a unified data pipeline.

The stack was developed incrementally:

1. **OpenAPI specification** (2016): Contract-first API documentation and code generation.
2. **Search engine** (April 2018): Full-text indexing of logs, documents, and product catalogs.
3. **Telemetry pipeline** (June 2018): Metrics collection, aggregation, and alerting.
4. **Session recording** (September 2018): DOM-level replay of user sessions.

Together, these components provide the observability layer for all Hanzo services.

# 2 Full-Text Search

## 2.1 Architecture

The Hanzo search engine (`hanzo/search`, 16,000 commits) provides full-text search, faceted filtering, and relevance ranking across structured and unstructured data. It indexes commerce data (products, orders, customers), application logs, and documentation.

## 2.2 Indexing Pipeline

Documents enter the search engine through a write-ahead log:

1. **Ingestion**: Documents are received via HTTP or the PubSub event bus.
2. **Analysis**: Text fields are tokenized, normalized (lowercasing, stemming), and analyzed for language detection.
3. **Indexing**: Inverted indices are constructed and flushed to segment files.
4. **Merge**: Background processes merge small segments into larger ones to maintain query performance.

## 2.3 Query Language

The search API supports structured queries combining full-text matching, range filters, and Boolean operators:

Listing 1: Search query example.

```
POST /v1/search
{
  "index": "products",
  "query": {
    "bool": {
      "must": [
        {"match": {"name": "wireless headphones"}},
        {"range": {"price": {"gte": 50, "lte": 200}}}
      ],
      "filter": [
        {"term": {"category": "electronics"}}
      ]
    }
  },
  "size": 20,
  "from": 0
}
```

### 2.4 Relevance Scoring

Documents are scored using BM25:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})} \tag{1}$$

where $f(t, d)$ is the term frequency in document $d$, $|d|$ is the document length, avgdl is the average document length, and $k_1 = 1.2$, $b = 0.75$ are tuning parameters.

### 2.5 Multi-Tenant Isolation

Each tenant's data is stored in a separate index namespace. Cross-tenant queries are prevented at the routing layer by injecting the tenant identifier (derived from the authenticated API key) into every query as a mandatory filter.

## 3 Telemetry and Metrics

### 3.1 Architecture

The telemetry pipeline (`hanzo/telemetry`, 8,200 commits) collects metrics from all Hanzo services, aggregates them into time series, and provides querying and alerting capabilities.

### 3.2 Data Model

Metrics follow a labeled time series model:

$$\text{timeseries} = (\text{name}, \text{labels}, [(t_1, v_1), (t_2, v_2), \ldots]) \tag{2}$$

where name is the metric name, labels is a set of key-value pairs, and $(t_i, v_i)$ are timestamp-value pairs.

Four metric types are supported:

- **Counter**: Monotonically increasing value (e.g., total requests).
- **Gauge**: Point-in-time measurement (e.g., active connections).

- **Histogram**: Distribution of values in configurable buckets (e.g., request latency).
- **Summary**: Pre-computed quantiles over a sliding time window.

### 3.3 Collection

Services expose metrics at a `/metrics` endpoint in the Prometheus exposition format. The telemetry collector scrapes these endpoints at configurable intervals (default: 15 seconds) and writes the data to the time series storage backend.

### 3.4 Alerting

Alert rules are defined as expressions over metric queries:

Listing 2: Alert rule definition.

```
groups:
- name: api-health
  rules:
  - alert: HighErrorRate
    expr: |
      rate(http_requests_total{status=~"5.."}[5m])
      / rate(http_requests_total[5m]) > 0.05
    for: 5m
    annotations:
      summary: "Error rate above 5%"
```

Alerts are routed to notification channels (PagerDuty, Slack, email) through a deduplication and grouping layer that prevents alert storms.

## 4 Session Recording

### 4.1 Architecture

The session recording system (`hanzo/rrweb`, September 2018, 1,600 commits) captures DOM mutations and user interactions in the browser, producing a replayable recording of user sessions.

### 4.2 Capture Mechanism

The recording library instruments the browser DOM using `MutationObserver`:

1. **Initial snapshot**: A serialized representation of the full DOM tree at session start.
2. **Incremental mutations**: Additions, removals, and attribute changes to DOM nodes.
3. **Input events**: Keystrokes (masked for sensitive fields), mouse movements, clicks, and scrolls.
4. **Network events**: API request/response metadata (no response bodies by default).

Events are batched and transmitted to the telemetry backend via the PubSub layer at one-second intervals.

### 4.3 Privacy Controls

Sensitive data is handled through configurable masking rules:

- **Input masking**: Password fields and elements with a `data-rrweb-mask` attribute have their values replaced with asterisks.
- **Block elements**: Elements with `data-rrweb-block` are replaced with placeholder rectangles in the recording.
- **Network filtering**: Request/response bodies are not captured. URL parameters can be stripped via regex rules.

### 4.4 Replay

Recordings are replayed in the Hanzo Insights dashboard by reconstructing the DOM from the initial snapshot and applying incremental mutations in timestamp order. The replay engine supports speed control (0.5x to 8x), timeline scrubbing, and event highlighting.

## 5 Event Analytics

### 5.1 Event Tracking

The analytics layer collects custom events from client and server applications:

Listing 3: Event tracking API.

```
hanzo.track('product_viewed', {
  productId: 'prod_123',
  category: 'electronics',
  price: 99.99,
  source: 'search_results'
});
```

Events are enriched with device, session, and user context before ingestion into the analytics store.

### 5.2 Funnel Analysis

Funnels are defined as ordered sequences of events. The funnel engine computes conversion rates between each step:

$$\text{conversion}(s_i \to s_{i+1}) = \frac{|\{u : u \in U(s_i) \cap U(s_{i+1})\}|}{|U(s_i)|} \tag{3}$$

where $U(s_i)$ is the set of users who completed step $s_i$ within the defined time window.

### 5.3 Cohort Analysis

Users are grouped into cohorts by acquisition date, behavior, or custom properties. Retention is measured as the fraction of a cohort that returns in subsequent time periods:

$$\text{retention}(c, t) = \frac{|\{u \in c : \text{active}(u, t)\}|}{|c|} \tag{4}$$

## 6 OpenAPI Specification

The OpenAPI specification layer (2016) defines the contract for all Hanzo APIs. Specifications are written in OpenAPI 3.0 format and used for:

- **Documentation**: Auto-generated API reference documentation.
- **Code generation**: Client SDKs in TypeScript, Go, and Python.
- **Validation**: Runtime request/response validation against the schema.
- **Testing**: Contract tests that verify API responses match the specification.

## 7 Integration

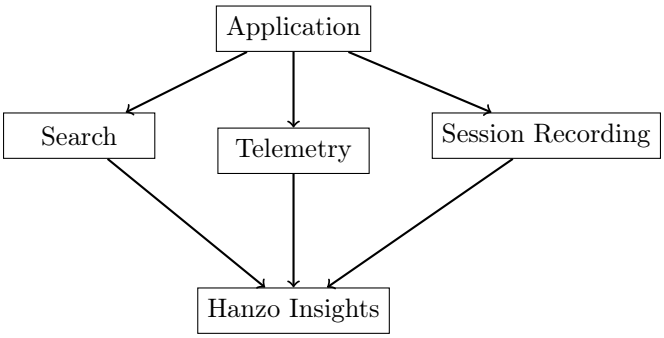


Figure 1: Observability data flow into Hanzo Insights.

All three data streams (search queries, metrics, session recordings) converge in the Hanzo Insights dashboard. Operators can correlate a performance anomaly (telemetry) with the user experience (session recording) and the search queries that led to the affected page.

## 8 Operational Scale

Table 1: Observability stack operational metrics (2018).

| **Metric** | **Value** |
|---|---:|
| Search index size | 2.4 TB |
| Queries per second (search) | 12,000 |
| Metrics time series (active) | 1,200,000 |
| Scrape interval | 15 s |
| Session recordings per day | 85,000 |
| Mean recording size | 340 KB |
| Event ingestion rate | 45,000 events/s |

## 9 Conclusion

The Hanzo observability stack provides full-text search, metrics collection, session recording, and event analytics as a unified platform. The system evolved from an OpenAPI specification

layer (2016) into a comprehensive observability solution by late 2018. Privacy-first design, multi-tenant isolation, and integration with the Hanzo PubSub event bus ensure that observability data is collected, stored, and queried without exposing sensitive user information across tenant boundaries. This infrastructure became the foundation for Hanzo Insights, the analytics product offered to Hanzo platform tenants.

## References


[1] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[2] Prometheus Authors. Prometheus: Monitoring system and time series database. `https://prometheus.io`, 2015.

[3] Y. Yin. rrweb: Record and replay the web. `https://rrweb.io`, 2018.

[4] OpenAPI Initiative. OpenAPI Specification 3.0. `https://openapis.org`, 2016.